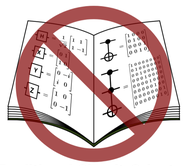
Qwerty

Students: Akshay Daftardar, Raghav Vikramprabhu
Mentors: Austin Adams

Georgia Tech
Open Source Program Office

Project Overview

Qwerty is a quantum programming language with a higher level of abstraction than quantum circuits.



Traditional Circuit-Oriented Quantum Metaprogramming

Programming in Qwerty

This summer, we will make the Qwerty compiler and REPL support the full Qwerty language.

Reimagined Representations of Qwerty Basis Vectors (Akshay): This project rewrote the MLIR representation as a tree and updated all upstream and downstream dependencies, as well as tests, ensuring that the example scripts functioned as before.

Basis translation & REPL (Raghav): Connected QWERTY parser to code that simulates Qubits, with this, we can simulate code **in real time** (jupyter notebook!)

Goals and Milestones 1

M1 — done: Recursive BasisVectorTreeAttr in MLIR: definition, verifier, parser/printer, unit tests.
M2 — done: Full downstream cutover (~50 sites across the C++ dialect, conversion pass, C API, Rust bindings, and front-end lowering), all test suites green.

Goals and Milestones 2

M1 — Bug-fix REPL interpreter to properly process Tensors and Singular Qubits + Bug-fix floating point bug on qubit equality to 0 - done
M2 — Write Basis Translation Algorithm Paper - done
M3 - Implement Basis Translation Algorithm - done

Highlights and Accomplishments: Reimagined Representations of Qwerty Basis Vectors

Before

```
list:{“|m>”}
```

After

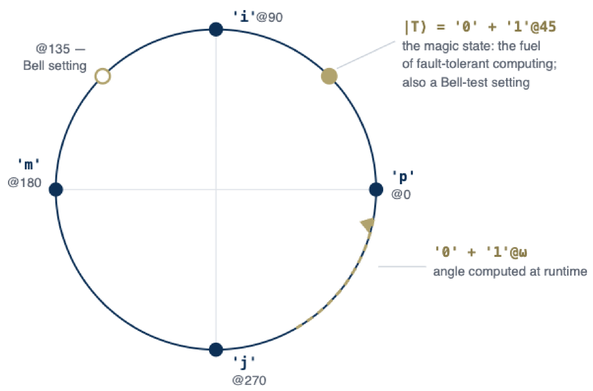
```
list:{
  <UniformVectorSuperpos [
    #qwerty.vectree<ZeroVector []>,
    #qwerty.vectree<VectorTilt tilt
180.0 [
      #qwerty.vectree<OneVector []>
    ]>
  ]>
}
```

From the User’s Perspective

```
{‘0’+‘1’@45, ‘0’-’1’@45}.measure
```

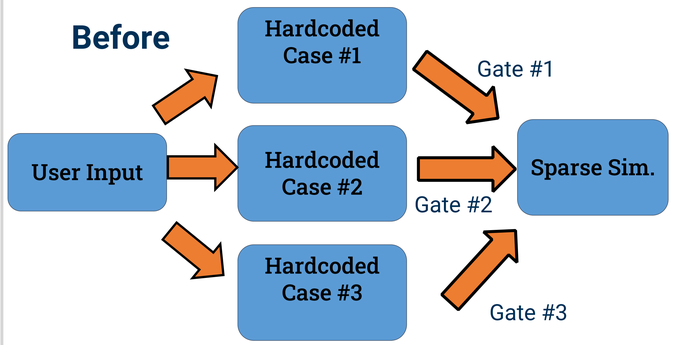
Non-trivial non-Pauli superpositions are now unlocked.

The Implications

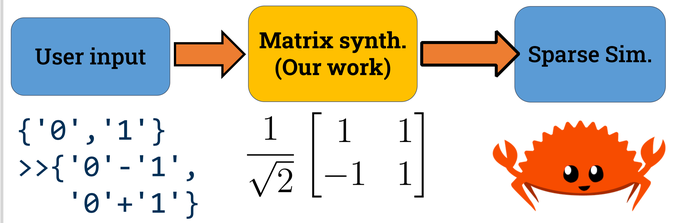


Highlights and Accomplishments: Elementary Basis Translation Algorithm

Before

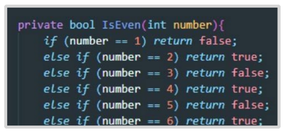


After



Analogy

90% Less Code
∞ More Utility



```
private bool IsEven(int number){
    if (number == 1) return false;
    else if (number == 2) return true;
    else if (number == 3) return false;
    else if (number == 4) return true;
    else if (number == 5) return false;
    else if (number == 6) return true;
```

is_even = lambda x: x % 2 == 0

From the User’s Perspective

```
>>> '0' | {'0','1'} >> {'0'-'1', '0'+'1'}
'0'-'1'
```

Programmers can run any basis translation in the Qwerty REPL.

Open Source Outcomes

- Infinite more quantum state capabilities in interpreter
- IR representation of all quantum states in QWERTY pipeline
- Optimized QWERTY compiler for Quantum Hardware

Future Work

- Allow for non-Pauli synthesis
- Cleanup primitive basis code
- Optimized Basis Translation Unitary Generation (Fast Matrix Multiplication)

Georgia Tech